Quero desenvolver um software para modelagem de banco de dados, com a facilidade de desenho do MySQL Workbench, ...

21:18

Para criar uma ferramenta de modelagem de banco de dados web, dockerizada e que suporte múltiplos SGDBs (como PostgreSQL, MySQL, SQL Server e Oracle), você precisa focar em tecnologias que resolvam três grandes pilares: **uma interface visual interativa (Frontend), gerenciamento de esquemas e dialetos abstratos (Backend) e portabilidade de ambiente (DevOps)**.

Abaixo está a pilha de tecnologias recomendada para construir esse ecossistema:

---

1. Frontend: O "Canvas" de Desenho Interativo

Para entregar a experiência de arrastar tabelas e ligar chaves estrangeiras de forma fluida no navegador, você precisará de bibliotecas de diagramação robustas.

- **Framework Principal:** React ou Vue.js. Eles ajudam a controlar o estado complexo do diagrama de tabelas (colunas, tipos de dados, chaves).
- **Biblioteca de Diagramação (O Core):**
  - **React Flow ou Vue Flow:** São as melhores opções de código aberto atuais. Permitem criar nós customizados (que seriam suas tabelas) e gerenciar conexões (relacionamentos) de forma ultra responsiva.
  - **JointJS / Rappid:** Excelente alternativa comercial ou open source focada puramente em diagramas de engenharia e gráficos interativos.

2. Backend: Suporte Multibanco e Abstração

O seu backend precisa traduzir as ações do usuário na tela em comandos SQL válidos para diferentes bancos.

- **Camada de Abstração de Banco de Dados (Drivers/Query Builders):**
  - **Knex.js:** Essencial. Ele é um query builder agnóstico que traduz JavaScript/TypeScript em dialetos específicos de múltiplos bancos (PostgreSQL, MySQL, SQLite, MSSQL, Oracle).
  - **Prisma ou TypeORM:** Úteis se você quiser trabalhar com uma API baseada em modelos estruturados, embora ferramentas que geram raw SQL customizado (como o Knex) ofereçam mais flexibilidade para manipulação de metadados dinâmicos.

3. Dockerização e Infraestrutura

Como o ambiente precisa ser totalmente dockerizado, você deve isolar a aplicação e simplificar o desenvolvimento.

- **Docker:** Para empacotar o Frontend e o Backend em containers individuais (Multi-stage builds para otimizar o tamanho da imagem web).
- **Docker Compose:** Para orquestrar o ambiente local. O seu arquivo `docker-compose.yml` deve subir:
  1. O container do seu app (Front + Back).
  2. Containers locais de teste de bancos de dados (um container Postgres, um MySQL, etc.) para que seu software possa testar a geração automática e engenharia reversa diretamente neles.

Comparativo de Arquitetura

Camada	Tecnologia Recomendada	Função Principal
Interface	React + React Flow	Criação visual das tabelas, arrastar blocos, ligar PKs a FKs.
Servidor	Node.js (TypeScript)	Processa a estrutura JSON gerada pelo front e gerencia conexões.
Abstração	Knex.js	Traduz o modelo conceitual em scripts SQL específicos de cada banco.

1. **No Frontend:** O usuário cria duas tabelas usando **React Flow** e arrasta uma linha conectando o ID de uma à outra. O front gera um payload JSON que descreve essa estrutura.
2. **No Backend:** O **Node.js** recebe esse JSON. Com base no banco alvo escolhido pelo usuário (ex: PostgreSQL), ele roda o motor de tradução (usando lógica própria em conjunto com o **Knex.js**) para converter o formato visual em um script DDL (`CREATE TABLE...`).
3. **Execução/Exportação:** O backend disponibiliza o download do arquivo `.sql` ou executa direto em um banco conectado através dos drivers nativos dockerizados.

Para te ajudar a dar o primeiro passo, você prefere que eu monte um exemplo prático de **como estruturar o arquivo Docker Compose** para esse ambiente ou um **exemplo inicial de código** integrando o gerador de tabelas no backend?

Gostaria de ver um exemplo de código do frontend para entender melhor o funcionamento do react + react flow.

21:24

Para entender como o **React Flow** funciona na prática, **imagine que cada tabela do banco de dados será um Nó Customizado (Custom Node) e os relacionamentos serão as Linhas de Conexão (Edges)**.

Abaixo, montei um exemplo prático e simplificado de como estruturar essa interface. O código define um nó customizado para representar uma tabela e renderiza o painel visual onde você pode arrastar e conectar as chaves.

1. Criando o Componente da Tabela (Nó Customizado)

Primeiro, criamos como uma tabela deve se parecer visualmente. Ela precisa de pontos de encaixe (**Handles**) para que o usuário consiga ligar as chaves estrangeiras.

jsx

```
  return (
    <div style={{
      background: '#fff',
      border: '2px solid #4a5568',
      borderRadius: '6px',
      minWidth: '200px',
      boxShadow: '0 4px 6px rgba(0,0,0,0.1)'
    }}>
      {/* Nome da Tabela */}
      <div style={{
        background: '#4a5568',
        color: '#fff',
        padding: '6px 12px',
        fontWeight: 'bold',
        borderTopLeftRadius: '4px',
        borderTopRightRadius: '4px'
      }}>
        {data.tableName}
      </div>

      {/* Lista de Colunas */}
      <div style={{ padding: '8px' }}>
        {data.columns.map((col, index) => (
          <div key={index} style={{
            display: 'flex',
            justifyContent: 'space-between',
            fontSize: '13px',
            margin: '4px 0'
          }}>
            <span style={{ fontWeight: col.isPK ? 'bold' : 'normal' }}>
              {col.name} {col.isPK && '🔑'}
            </span>
            <span style={{ color: '#718096' }}>{col.type}</span>
          </div>
        ))}
      </div>

      {/* Pontos de conexão para Chaves Estrangeiras (Esquerda e Direita) */}
      <Handle type="target" position={Position.Left} style={{ background: '#31
      <Handle type="source" position={Position.Right} style={{ background: '#3
    </div>
```

2. Montando o Canvas Principal (O Diagrama)

Agora, conectamos o React Flow ao estado do React. É aqui que definimos quais tabelas existem inicialmente e capturamos as conexões criadas pelo usuário.

jsx

```jsx
// App.jsx
import React, { useState, useCallback } from 'react';
import ReactFlow, {
  Background,
  Controls,
  applyEdgeChanges,
  applyNodeChanges,
  addEdge
} from 'reactflow';
import 'reactflow/dist/style.css'; // Importante: carrega os estilos base da b

import { TableNode } from './TableNode';

// Registramos nosso componente customizado no React Flow
const nodeTypes = { tableNode: TableNode };

// Dados iniciais de exemplo (Duas tabelas: Users e Orders)
const initialNodes = [
  {
    id: 'users',
    type: 'tableNode', // Usa o tipo que registramos acima
    position: { x: 100, y: 100 },
    data: {
      tableName: 'users',
      columns: [
        { name: 'id', type: 'INT', isPK: true },
        { name: 'name', type: 'VARCHAR(255)', isPK: false },
        { name: 'email', type: 'VARCHAR(255)', isPK: false }
      ]
    }
  },
  {
```

```
        { name: 'id', type: 'INT', isPK: true },
        { name: 'user_id', type: 'INT', isPK: false }, // Chave estrangeira co
        { name: 'total', type: 'DECIMAL', isPK: false }
      ]
    }
  }
];

const initialEdges = [];

export default function App() {
  const [nodes, setNodes] = useState(initialNodes);
  const [edges, setEdges] = useState(initialEdges);

  // Funções padrão do React Flow para permitir arrastar e selecionar elemento
  const onNodesChange = useCallback((changes) => setNodes((nds) => applyNodeCh
  const onEdgesChange = useCallback((changes) => setEdges((eds) => applyEdgeCh

  // Evento disparado quando o usuário arrasta uma linha de uma tabela até a o
  const onConnect = useCallback((connection) => {
    console.log("Nova FK Criada!", connection);
    // Aqui você pode disparar modais para o usuário configurar se a relação é
    setEdges((eds) => addEdge({ ...connection, animated: true, style: { stroke
  }, []);

  // Função para simular o envio do esquema atual para o seu Backend dockeriza
  const exportSchemaToBackend = () => {
    const payload = { nodes, edges };
    console.log("JSON enviado ao Backend:", JSON.stringify(payload, null, 2));
    // Ex: axios.post('/api/generate-sql', payload);
  };

  return (
    <div style={{ width: '100vw', height: '100vh', background: '#f7fafc' }}>
      <div style={{ padding: '10px', position: 'absolute', zIndex: 4, display:
        <button onClick={exportSchemaToBackend} style={{ padding: '8px 12px',
          Gerar SQL no Backend
        </button>
      </div>

      <ReactFlow
```

```
          fitView
        >
          <Background color="#cbd5e0" gap={16} />
          <Controls />
        </ReactFlow>
      </div>
    );
}
```

Use o código com cuidado.

Como esse JSON se traduz em código

Quando você clica no botão "Gerar SQL", o React Flow entrega uma estrutura limpa de dados. O backend receberá uma lista de conexões indicando que a tabela `orders` se conecta à tabela `users` (`source: "users"`, `target: "orders"`). Seu interpretador em Node.js lerá esse mapa para injetar a restrição `FOREIGN KEY (user_id) REFERENCES users(id)` dinamicamente!

Deseja avançar vendo **como o backend em Node.js (TypeScript)** receberia esse formato de JSON para montar dinamicamente a string SQL, ou prefere focar em como estruturar a **arquitetura do Docker** para expor esse app web?

Quero ver como receber o JSON.

21:26

Para processar esse JSON no **Backend**, você criará uma rota (geralmente uma requisição `POST`) que lê a estrutura enviada pelo React Flow. O papel do backend é varrer a lista de nós (tabelas) e conexões (relacionamentos) para montar o script SQL correspondente ao banco escolhido.

Abaixo está um exemplo prático utilizando **Node.js (TypeScript)** com **Express** e o query builder **Knex.js** para simular como receber esses dados e gerar o DDL (Data Definition Language)

`mysql`, `postgres` ou `sqlite3`).

typescript

```typescript
// server.ts
import express, { Request, Response } from 'express';
import cors from 'cors';
import knex from 'knex';

const app = express();
app.use(cors());
app.use(express.json()); // Permite receber o payload JSON do React Flow

// Interfaces para tipar o JSON recebido do Frontend
interface Column {
  name: string;
  type: string;
  isPK: boolean;
}

interface TableNode {
  id: string;
  data: {
    tableName: string;
    columns: Column[];
  };
}

interface ConnectionEdge {
  source: string; // ID da tabela pai (ex: 'users')
  target: string; // ID da tabela filho (ex: 'orders')
  sourceHandle?: string; // (Opcional) Id da coluna PK se mapeado no front
  targetHandle?: string; // (Opcional) Id da coluna FK se mapeado no front
}

interface SchemaPayload {
  targetDatabase: 'mysql' | 'postgres' | 'sqlite3'; // O banco selecionado na
  nodes: TableNode[];
  edges: ConnectionEdge[];
}
```

```
    // Instanciamos um Knex virtual (sem conexão real com o banco) apenas para
    const qb = knex({ client: targetDatabase });
    let sqlOutput = `-- Gerado automaticamente para ${targetDatabase.toUpperCa

    // 1. Gerar as Tabelas e suas Colunas
    for (const node of nodes) {
      const { tableName, columns } = node.data;

      // Cria a definição da tabela usando o Schema Builder do Knex
      const tableSql = await qb.schema.createTable(tableName, (table) => {
        columns.forEach((col) => {
          // Define dinamicamente o tipo da coluna com base no dialeto escolhi
          let columnDefinition;

          if (col.type.toUpperCase().startsWith('VARCHAR')) {
            columnDefinition = table.string(col.name, 255);
          } else if (col.type.toUpperCase() === 'INT') {
            columnDefinition = table.integer(col.name);
          } else if (col.type.toUpperCase() === 'DECIMAL') {
            columnDefinition = table.decimal(col.name, 10, 2);
          } else {
            columnDefinition = table.specificType(col.name, col.type);
          }

          // Define se é Chave Primária
          if (col.isPK) {
            columnDefinition.primary();
          }
        });

        // 2. Mapear e injetar Chaves Estrangeiras vinculadas a esta tabela
        // Procura se existe alguma linha ligando outra tabela (source) até es
        const foreignKeys = edges.filter(edge => edge.target === node.id);

        foreignKeys.forEach(fk => {
          // Localiza o nó pai para descobrir o nome real da tabela de origem
          const parentNode = nodes.find(n => n.id === fk.source);
          if (parentNode) {
            const parentTableName = parentNode.data.tableName;

            // Convenção simplificada: assume que o campo local aponta para o
```

```
                .onDelete('CASCADE');
            }
          });
      }).toSQL(); // Converte a estrutura lógica criada acima em uma string SQ

      // Agrupa os comandos SQL gerados (retorna um array de queries)
      tableSql.forEach(query => {
        sqlOutput += `${query.sql};\n`;
      });
      sqlOutput += '\n';
    }

    // Retorna o script SQL final pronto para o Frontend exibir na tela
    res.json({ sql: sqlOutput });

  } catch (error: any) {
    console.error(error);
    res.status(500).json({ error: 'Erro ao processar o esquema e gerar o SQL.'
  }
});

app.listen(3001, () => {
  console.log('Servidor rodando na porta 3001 🚀');
});
```

Use o código com cuidado.

O Resultado Prático

Se o frontend enviar as tabelas `users` e `orders` ligadas entre si, e marcar o `targetDatabase` como `postgres`, a resposta que o backend devolverá em formato JSON será exatamente essa:

json

```
{
  "sql": "-- Gerado automaticamente para POSTGRES\n\nCREATE TABLE \"users\" (\
```

Caso o usuário alterne na interface para `mysql`, o Knex automaticamente altera as aspas duplas por crases (`) e ajusta pequenos detalhes de sintaxe específicos do motor do MySQL, poupando você de escrever dezenas de estruturas `if/else` manuais para cada banco do mercado.

Agora que passamos pela estrutura do **Frontend** e do **Backend**, o que faz mais sentido para o seu próximo passo?

- Criar a estrutura do **Dockerfile** e **Docker Compose** para envelopar e rodar o projeto de forma isolada?
- Entender como capturar no React Flow **a coluna exata** que foi clicada (usando múltiplos Handles por nó) em vez de ligar apenas a caixa da tabela inteira?

Criar a estrutura do Dockerfile e Docker Compose

21:44

Para rodar esse ecossistema completo de forma isolada, utilizaremos uma arquitetura multi-container. **O objetivo é ter um container para o Frontend (React), um para o Backend (Node.js) e containers adicionais para os Bancos de Dados locais**, permitindo que seu software teste a geração de esquemas ao vivo.

Aqui está a estrutura de arquivos necessária para dockerizar a aplicação.

---

1. Dockerfile do Frontend (React)

Crie este arquivo dentro da pasta do seu frontend (ex: `./frontend/Dockerfile`). Ele utiliza **multi-stage build** para compilar o app e servir os arquivos estáticos de forma leve usando o Nginx.

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Estágio 2: Servidor de Produção leve com Nginx
FROM nginx:alpine
# Copia os arquivos buildados do estágio anterior para o diretório do Nginx
COPY --from=build /app/dist /usr/share/nginx/html
# Copia uma configuração customizada para o Nginx lidar com rotas do React (SP
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Use o código com cuidado.

(Nota: Lembre-se de criar um arquivo simples `nginx.conf` na pasta do front para redirecionar todas as requisições para o `index.html`, evitando erros de 404 ao atualizar a página).

---

2. Dockerfile do Backend (Node.js + TypeScript)

Crie este arquivo dentro da pasta do seu backend (ex: `./backend/Dockerfile`). Ele instala as dependências, compila o TypeScript e roda a aplicação.

dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
# Compila o TypeScript para JavaScript (gera a pasta /dist ou similar)
RUN npm run build
EXPOSE 3001
CMD ["npm", "run", "start"]
```

3. O Arquivo Docker Compose Principal

Na **raiz do projeto**, crie o arquivo `docker-compose.yml`. Ele vai orquestrar o Frontend, o Backend e subir instâncias do **PostgreSQL** e **MySQL** automaticamente na mesma rede para que o seu backend consiga se comunicar com eles.

yaml

```yaml
version: '3.8'

services:
  # Serviço do Frontend (React Flow Interface)
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "80:80"
    depends_on:
      - backend
    networks:
      - db-designer-network

  # Serviço do Backend (Node.js Compiler)
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "3001:3001"
    environment:
      - PORT=3001
      # Variáveis para o backend se conectar aos bancos de teste, se necessári
      - POSTGRES_HOST=postgres-test
      - MYSQL_HOST=mysql-test
    depends_on:
      - postgres-test
      - mysql-test
    networks:
      - db-designer-network
```

```
      POSTGRES_PASSWORD: designer_password
      POSTGRES_DB: test_db
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - db-designer-network

  # Banco de Dados de Teste: MySQL
  mysql-test:
    image: mysql:8.0
    container_name: mysql-test
    environment:
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_DATABASE: test_db
      MYSQL_USER: designer_user
      MYSQL_PASSWORD: designer_password
    ports:
      - "3306:3306"
    volumes:
      - mysqldata:/var/lib/mysql
    networks:
      - db-designer-network

volumes:
  pgdata:
  mysqldata:

networks:
  db-designer-network:
    driver: bridge
```

Use o código com cuidado.

Como essa engrenagem roda junta?

1. **Rede Isolada** (`db-designer-network`): Todos os containers ficam na mesma rede virtual

Postgres locais não sejam apagados.

3. **Comando para Rodar:** Para subir todo o ambiente de uma vez só, basta abrir o terminal na raiz do projeto e rodar:

```bash
docker compose up --build
```

Use o código com cuidado.

A partir desse ponto, ao acessar `http://localhost`, você verá a sua interface do React Flow pronta para desenhar as tabelas.

Agora que toda a infraestrutura está desenhada, como você gostaria de prosseguir?

- Podemos detalhar a **estratégia de engenharia reversa** (como fazer o backend ler um banco real via Docker e gerar o JSON para o front).
- Ou quer ajustar o React Flow para tratar **múltiplos Handles (pontos de encaixe)** por linha/coluna da tabela?

Quero ajustar o React Flow para tratar múltiplos Handles (pontos de encaixe) por linha/coluna da tabela

22:10

Para permitir que o usuário conecte **uma coluna específica de uma tabela à coluna de outra tabela** (como uma chave estrangeira real), precisamos evoluir o nosso componente de nó customizado (`TableNode`).

No React Flow, para colocar múltiplos pontos de encaixe em um único nó, você só precisa garantir duas regras:

Aqui está a implementação prática dessa evolução:

1. Atualizando o Componente da Tabela com Múltiplos Handles

Vamos alterar o `TableNode.jsx` para que cada linha da tabela se torne um container relativo com seus próprios pontos de entrada (`target`) e saída (`source`).

jsx

```jsx
// TableNode.jsx
import React from 'react';
import { Handle, Position } from 'reactflow';

export function TableNode({ data }) {
  return (
    <div style={{
      background: '#fff',
      border: '2px solid #4a5568',
      borderRadius: '6px',
      minWidth: '240px',
      boxShadow: '0 4px 6px rgba(0,0,0,0.1)',
      fontFamily: 'sans-serif'
    }}>
      {/* Título/Nome da Tabela */}
      <div style={{
        background: '#4a5568',
        color: '#fff',
        padding: '8px 12px',
        fontWeight: 'bold',
        borderTopLeftRadius: '4px',
        borderTopRightRadius: '4px',
        fontSize: '14px'
      }}>
        {data.tableName}
      </div>

      {/* Corpo da Tabela (Linhas/Colunas) */}
      <div style={{ padding: '4px 0' }}>
        {data.columns.map((col) => {
          // Criamos um ID único para cada handle combinando a tabela e a colu
```

```
            position: 'relative', // CRUCIAL para o posicionamento correto
            display: 'flex',
            justifyContent: 'space-between',
            alignItems: 'center',
            padding: '6px 16px',
            fontSize: '13px',
            borderBottom: '1px solid #edf2f7'
          }}
        >
          {/* Handle de Entrada (Esquerda) - O ID único diz ao React Flow
          <Handle
            type="target"
            position={Position.Left}
            id={`${handleId}-target`}
            style={{
              background: '#a0aec0',
              width: '8px',
              height: '8px',
              left: '-5px' // Move um pouco para fora da borda
            }}
          />

          {/* Texto descritivo da coluna */}
          <span style={{ fontWeight: col.isPK ? 'bold' : 'normal', marginR
            {col.name} {col.isPK && '🔑'}
          </span>
          <span style={{ color: '#718096', fontSize: '11px' }}>{col.type}<

          {/* Handle de Saída (Direita) */}
          <Handle
            type="source"
            position={Position.Right}
            id={`${handleId}-source`}
            style={{
              background: '#3182ce',
              width: '8px',
              height: '8px',
              right: '-5px' // Move um pouco para fora da borda
            }}
          />
        </div>
```

Use o código com cuidado.

Como as conexões chegam no Backend agora?

Quando o usuário arrastar uma linha ligando a coluna `user_id` da tabela `orders` até a coluna `id` da tabela `users`, a função `onConnect` do React Flow vai gerar um objeto de conexão muito mais rico.

O JSON gerado e enviado para o seu backend ficará assim:

json

```json
{
  "source": "orders",
  "target": "users",
  "sourceHandle": "orders.user_id-source",
  "targetHandle": "users.id-target"
}
```

Use o código com cuidado.

Ajuste sugerido para o seu Backend em Node.js

Com esses IDs específicos de coluna chegando em `sourceHandle` e `targetHandle`, você pode refatorar o trecho do backend (que mostramos anteriormente) para não precisar mais "adivinhar" o nome da coluna estrangeira.

Em vez de fazer `const localFkColumn = \`${parentTableName}_id``, você pode extrair o nome exato diretamente da string enviada:

typescript

```typescript
// No seu backend Node.js:
const sourceColumnName = fk.sourceHandle.split('.')[1].replace('-source', '');
```

Use o código com cuidado.

Deseja expandir esse comportamento para que o usuário possa **adicionar ou editar novas colunas dinamicamente** clicando em um botão dentro do próprio nó, ou prefere focar em como implementar a lógica de **engenharia reversa** para ler um banco Docker já existente e desenhar as tabelas na tela?

Primeiro, adicionar ou editar novas colunas dinamicamente clicando em um botão dentro do próprio nó.

22:11

Para permitir que o usuário crie e edite colunas diretamente na interface visual, precisamos dar ao nosso nó customizado (`TableNode`) a capacidade de gerenciar interações e alterar o estado global de nós do **React Flow**.

A forma mais elegante e performática de fazer isso é criando um botão de "Adicionar Coluna" e inputs de edição rápidos diretamente no nó.

Abaixo está o código atualizado do seu componente de tabela para dar essa autonomia ao usuário:

1. Atualizando o `TableNode.jsx` com Interações Dinâmicas

Para atualizar o estado de um nó de dentro dele mesmo, o React Flow fornece um hook muito útil chamado `useReactFlow`. Com ele, podemos atualizar os dados (`data.columns`) da tabela clicada.

jsx

```
// TableNode.jsx
```

```
// Função para adicionar uma nova coluna padrão na tabela atual
const addColumn = () => {
  setNodes((nodes) =>
    nodes.map((node) => {
      if (node.id === id) {
        // Cria uma nova coluna com nome e tipo genéricos para o usuário edi
        const newColumn = {
          name: `nova_coluna_${node.data.columns.length + 1}`,
          type: 'VARCHAR(255)',
          isPK: false,
        };
        return {
          ...node,
          data: {
            ...node.data,
            columns: [...node.data.columns, newColumn],
          },
        };
      }
      return node;
    })
  );
};

// Função para atualizar o nome ou tipo de uma coluna existente
const updateColumn = (index, field, value) => {
  setNodes((nodes) =>
    nodes.map((node) => {
      if (node.id === id) {
        const updatedColumns = [...node.data.columns];
        updatedColumns[index] = {
          ...updatedColumns[index],
          [field]: value,
        };
        return {
          ...node,
          data: { ...node.data, columns: updatedColumns },
        };
      }
      return node;
    })
```

```
      border: '2px solid #4a5568',
      borderRadius: '6px',
      minWidth: '280px', // Aumentado um pouco para acomodar os inputs
      boxShadow: '0 4px 6px rgba(0,0,0,0.1)',
      fontFamily: 'sans-serif'
    }}>
      {/* Cabeçalho da Tabela */}
      <div style={{
        background: '#4a5568',
        color: '#fff',
        padding: '8px 12px',
        fontWeight: 'bold',
        borderTopLeftRadius: '4px',
        borderTopRightRadius: '4px',
        fontSize: '14px',
        display: 'flex',
        justifyContent: 'space-between',
        alignItems: 'center'
      }}>
        <span>{data.tableName}</span>
        {/* Botão de Adicionar Coluna (+ Inline) */}
        <button
          onClick={addColumn}
          style={{
            background: '#48bb78',
            color: 'white',
            border: 'none',
            borderRadius: '4px',
            padding: '2px 8px',
            cursor: 'pointer',
            fontSize: '12px',
            fontWeight: 'bold'
          }}
        >
          + Coluna
        </button>
      </div>

      {/* Lista de Colunas Editáveis */}
      <div style={{ padding: '4px 0' }}>
        {data.columns.map((col, index) => {
```

```
            position: 'relative',
            display: 'flex',
            gap: '6px',
            alignItems: 'center',
            padding: '6px 12px',
            fontSize: '13px',
            borderBottom: '1px solid #edf2f7'
          }}
        >
          {/* Handle Target (Esquerda) */}
          <Handle
            type="target"
            position={Position.Left}
            id={`${handleId}-target`}
            style={{ background: '#a0aec0', width: '8px', height: '8px', l
          />

          {/* Botão de Chave Primária (Alternador) */}
          <button
            onClick={() => updateColumn(index, 'isPK', !col.isPK)}
            style={{
              background: 'none',
              border: 'none',
              cursor: 'pointer',
              fontSize: '14px',
              padding: 0
            }}
            title={col.isPK ? "Remover Chave Primária" : "Tornar Chave Pri
          >
            {col.isPK ? '🔑' : '⚪'}
          </button>

          {/* Input para Editar o Nome da Coluna */}
          <input
            type="text"
            value={col.name}
            onChange={(e) => updateColumn(index, 'name', e.target.value)}
            style={{
              border: '1px solid #cbd5e0',
              borderRadius: '3px',
              padding: '2px 4px',
```

```
              <select
                value={col.type}
                onChange={(e) => updateColumn(index, 'type', e.target.value)}
                style={{
                  border: '1px solid #cbd5e0',
                  borderRadius: '3px',
                  padding: '2px',
                  width: '90px',
                  fontSize: '11px',
                  background: '#fff'
                }}
              >
                <option value="INT">INT</option>
                <option value="VARCHAR(255)">VARCHAR(255)</option>
                <option value="TEXT">TEXT</option>
                <option value="DECIMAL">DECIMAL</option>
                <option value="BOOLEAN">BOOLEAN</option>
                <option value="TIMESTAMP">TIMESTAMP</option>
              </select>

              {/* Handle Source (Direita) */}
              <Handle
                type="source"
                position={Position.Right}
                id={`${handleId}-source`}
                style={{ background: '#3182ce', width: '8px', height: '8px', r
              />
            </div>
          );
        })}
      </div>
    </div>
  );
}
```

Use o código com cuidado.

2. O que muda no fluxo global do app?

Ao digitar uma nova letra no input ou alternar a chave (🔑), o React Flow intercepta a mudança

dinâmica.

Agora que o usuário consegue desenhar, criar e modificar tabelas e colunas livremente na tela, o que você prefere fazer a seguir?

- Criar a lógica de **Engenharia Reversa** no backend para que ele se conecte a um banco existente e envie as tabelas já prontas para o React Flow desenhar na tela.
- Adicionar a possibilidade de **criar novas tabelas do zero** arrastando um botão de um menu lateral para dentro do canvas.

Criar a lógica de Engenharia Reversa no backend

22:12

Para fazer a **Engenharia Reversa**, o backend precisa se conectar a um banco de dados real (que já subimos no nosso Docker Compose), ler o dicionário de dados do sistema (as tabelas do `information_schema`) e traduzir essa estrutura de metadados em um formato JSON que o React Flow consiga renderizar como nós e conexões.

Utilizaremos o **Knex.js** para realizar as consultas de metadados, aproveitando a conexão nativa com múltiplos drivers.

Abaixo está o exemplo prático de uma rota no backend Node.js (TypeScript) focada em extrair tabelas, colunas e relacionamentos de um banco **PostgreSQL** ou **MySQL**.

---

1. Criando a Rota de Engenharia Reversa no Backend

Adicione este endpoint ao seu arquivo de rotas do Express. Ele recebe as credenciais de conexão do usuário, varre as tabelas do banco e monta o payload esperado pelo React Flow.

typescript

```
interface ConnectionConfig {
  client: 'postgres' | 'mysql';
  host: string;
  user: string;
  password?: string;
  database: string;
  port: number;
}

app.post('/api/reverse-engineering', async (req: Request, res: Response) => {
  let dbConnection;

  try {
    const config: ConnectionConfig = req.body;

    // 1. Abre uma conexão real com o banco de dados do cliente usando as crec
    dbConnection = knex({
      client: config.client,
      connection: {
        host: config.host,
        user: config.user,
        password: config.password,
        database: config.database,
        port: config.port,
      }
    });

    // 2. Buscar todas as Tabelas e Colunas usando consultas nativas do Inform
    let columnsQuery = '';

    if (config.client === 'postgres') {
      columnsQuery = `
        SELECT
          c.table_name,
          c.column_name,
          c.data_type,
          EXISTS (
            SELECT 1 FROM information_schema.table_constraints tc
            JOIN information_schema.key_column_usage kcu ON tc.constraint_name
            WHERE tc.constraint_type = 'PRIMARY KEY'
```

```
    `;
  } else if (config.client === 'mysql') {
    columnsQuery = `
      SELECT
        TABLE_NAME as table_name,
        COLUMN_NAME as column_name,
        DATA_TYPE as data_type,
        IF(COLUMN_KEY = 'PRI', 1, 0) as is_pk
      FROM information_schema.COLUMNS
      WHERE TABLE_SCHEMA = '${config.database}'
      ORDER BY TABLE_NAME, ORDINAL_POSITION;
    `;
  }

  const { rows: rawColumns } = await dbConnection.raw(columnsQuery);

  // Para o MySQL, o retorno direto do .raw() pode variar dependendo do driv
  const columnsData = config.client === 'postgres' ? rawColumns : rawColumns

  // 3. Estruturar os dados coletados no formato de Nodes do React Flow
  const nodesMap: { [key: string]: any } = {};
  let initialX = 50;
  let initialY = 50;

  columnsData.forEach((row: any) => {
    const { table_name, column_name, data_type, is_pk } = row;

    if (!nodesMap[table_name]) {
      nodesMap[table_name] = {
        id: table_name,
        type: 'tableNode',
        // Organiza em cascata básica na tela para não sobrepor (o usuário r
        position: { x: initialX, y: initialY },
        data: {
          tableName: table_name,
          columns: []
        }
      };
      initialX += 320; // Espaçamento horizontal entre tabelas
      if (initialX > 1000) {
        initialX = 50;
```

```
      type: data_type.toUpperCase(),
      isPK: !!is_pk // Converte para booleano real
    });
  });

  const nodes = Object.values(nodesMap);

  // 4. Buscar os Relacionamentos (Chaves Estrangeiras) para montar as Edges
  let fkQuery = '';
  if (config.client === 'postgres') {
    fkQuery = `
      SELECT
        tc.table_name AS source_table,
        kcu.column_name AS source_column,
        ccu.table_name AS target_table,
        ccu.column_name AS target_column
      FROM information_schema.table_constraints AS tc
      JOIN information_schema.key_column_usage AS kcu ON tc.constraint_name
      JOIN information_schema.constraint_column_usage AS ccu ON ccu.constrai
      WHERE tc.constraint_type = 'FOREIGN KEY' AND tc.table_schema = 'public
    `;
  } else if (config.client === 'mysql') {
    fkQuery = `
      SELECT
        TABLE_NAME as source_table,
        COLUMN_NAME as source_column,
        REFERENCED_TABLE_NAME as target_table,
        REFERENCED_COLUMN_NAME as target_column
      FROM information_schema.KEY_COLUMN_USAGE
      WHERE CONSTRAINT_SCHEMA = '${config.database}'
        AND REFERENCED_TABLE_NAME IS NOT NULL;
    `;
  }

  const { rows: rawFks } = await dbConnection.raw(fkQuery);
  const fksData = config.client === 'postgres' ? rawFks : rawFks[0];

  // Mapeia chaves estrangeiras para o formato de Edges (Linhas de conexão)
  const edges = fksData.map((fk: any, index: number) => ({
    id: `edge-${index}`,
    source: fk.target_table, // Quem detém a PK original (Pai)
```

```
    // Envia a resposta final estruturada para o Frontend alimentar seu estadc
    res.json({ nodes, edges });

  } catch (error: any) {
    console.error('Erro na engenharia reversa:', error.message);
    res.status(500).json({ error: `Falha ao ler banco de dados: ${error.messag
  } finally {
    // Certifica-se de fechar o pool de conexões temporárias para não estourar
    if (dbConnection) await dbConnection.destroy();
  }
});
```

Use o código com cuidado.

---

2. Como o Frontend consome isso?

No seu frontend React, você pode criar uma tela ou modal de "Importar Banco Existente" que captura os dados de conexão. Ao receber o retorno do backend, basta injetar o resultado diretamente nos hooks de estado do React Flow.

javascript

```
// Exemplo de integração no App.jsx do Frontend
const handleImport = async () => {
  const connectionCredentials = {
    client: 'postgres',
    host: 'postgres-test', // Host configurado no Docker Compose
    user: 'designer_user',
    password: 'designer_password',
    database: 'test_db',
    port: 5432
  };

  const response = await axios.post('/api/reverse-engineering', connectionCred

  // Atualiza o estado do React Flow limpando o canvas anterior e desenhando c
```

Com este fluxo implementado, sua aplicação web agora é capaz de ler qualquer banco de dados rodando em sua infraestrutura Docker, interpretar as chaves primárias e estrangeiras e desenhar o diagrama ER completo de forma automatizada.

Para consolidar os recursos essenciais do projeto, o que gostaria de fazer agora?

- Adicionar a mecânica de **criar tabelas do zero arrastando itens de uma barra lateral (Drag and Drop)** para dentro do canvas do React Flow.
- Implementar um **sistema de auto-layout (como o Dagre ou Elkjs)** para organizar automaticamente os nós na tela ao rodar a engenharia reversa, evitando que as tabelas fiquem bagunçadas.

Adicionar a mecânica de criar tabelas do zero arrastando itens de uma barra lateral

22:13

Para criar essa funcionalidade de **Arrastar e Soltar (Drag and Drop)**, utilizaremos a API nativa de `dragstart` do HTML5 em conjunto com as funções internas do React Flow.

O fluxo funcionará assim: o usuário clica em um item em um painel lateral, arrasta para dentro do Canvas e, ao soltar o mouse, capturamos a **coordenada exata da tela** para injetar uma nova tabela no estado de nós.

Abaixo está o código atualizado do seu `App.jsx`, incluindo o painel lateral e a lógica de captura do evento de soltura (drop).

---

1. Atualizando o Canvas Principal (`App.jsx`)

Substitua o arquivo do seu painel principal por esta estrutura. Ela adiciona uma barra lateral à esquerda e usa a instância do React Flow para converter coordenadas de pixel do mouse em

```
import React, { useState, useCallback, useRef } from 'react';
import ReactFlow, {
  Background,
  Controls,
  applyEdgeChanges,
  applyNodeChanges,
  addEdge,
  useReactFlow,
  ReactFlowProvider
} from 'reactflow';
import 'reactflow/dist/style.css';

import { TableNode } from './TableNode';

const nodeTypes = { tableNode: TableNode };

function DiagramCanvas() {
  const reactFlowWrapper = useRef(null);
  const [nodes, setNodes] = useState([]);
  const [edges, setEdges] = useState([]);

  // O hook screenToFlowPosition ajuda a calcular onde o nó deve aparecer no z
  const { screenToFlowPosition } = useReactFlow();

  const onNodesChange = useCallback((changes) => setNodes((nds) => applyNodeCh
  const onEdgesChange = useCallback((changes) => setEdges((eds) => applyEdgeCh
  const onConnect = useCallback((connection) => {
    setEdges((eds) => addEdge({ ...connection, animated: true, style: { stroke
  }, []);

  // 1. Evento disparado quando o Canvas aceita o item que está sendo arrastad
  const onDragOver = useCallback((event) => {
    event.preventDefault();
    event.dataTransfer.dropEffect = 'move';
  }, []);

  // 2. Evento disparado no momento exato em que o usuário solta o mouse no Ca
  const onDrop = useCallback(
    (event) => {
      event.preventDefault();

      // Recupera o tipo de nó que definimos no painel lateral
```

```
      }

      // Calcula a posição do mouse adaptada ao zoom e pan atuais do canvas
      const position = screenToFlowPosition({
        x: event.clientX,
        y: event.clientY,
      });

      // Cria a estrutura padrão de uma tabela vazia com uma chave primária pa
      const tableId = `tabela_${Date.now()}`;
      const newNode = {
        id: tableId,
        type,
        position,
        data: {
          tableName: tableId,
          columns: [
            { name: 'id', type: 'INT', isPK: true } // Toda tabela nasce com u
          ]
        },
      };

      setNodes((nds) => nds.concat(newNode));
    },
    [screenToFlowPosition]
  );

  return (
    <div style={{ display: 'flex', width: '100vw', height: '100vh', fontFamily

      {/* BARRA LATERAL (Sidebar) */}
      <div style={{
        width: '240px',
        background: '#1a202c',
        color: '#fff',
        padding: '20px',
        boxShadow: '2px 0 5px rgba(0,0,0,0.1)',
        zIndex: 5
      }}>
        <h3 style={{ fontSize: '16px', marginBottom: '20px' }}>Componentes</h3
        <p style={{ fontSize: '12px', color: '#a0aec0', marginBottom: '15px' }
```

```
          onDragStart={(event) => {
            event.dataTransfer.setData('application/reactflow', 'tableNode');
            event.dataTransfer.effectAllowed = 'move';
          }}
          style={{
            padding: '12px',
            background: '#2d3748',
            border: '2px dashed #4a5568',
            borderRadius: '6px',
            cursor: 'grab',
            textAlign: 'center',
            fontWeight: 'bold',
            color: '#63b3ed'
          }}
        >
          🗂️ Nova Tabela
        </div>
      </div>

      {/* ÁREA DO CANVAS (React Flow) */}
      <div style={{ flexGrow: 1, height: '100%' }} ref={reactFlowWrapper}>
        <ReactFlow
          nodes={nodes}
          edges={edges}
          onNodesChange={onNodesChange}
          onEdgesChange={onEdgesChange}
          onConnect={onConnect}
          nodeTypes={nodeTypes}
          onDrop={onDrop}
          onDragOver={onDragOver}
          fitView
        >
          <Background color="#cbd5e0" gap={16} />
          <Controls />
        </ReactFlow>
      </div>
    </div>
  );
}

// O React Flow exige que funções que usem seus hooks (como useReactFlow) este
```

```
}
```

Use o código com cuidado.

Como testar na interface

1. Ao rodar o projeto, você verá o painel escuro à esquerda contendo o bloco "🗂️ **Nova Tabela**".
2. Clique, segure e arraste esse bloco para o meio do quadriculado cinza.
3. Ao soltar, uma nova tabela com o nome baseado no timestamp atual e uma coluna `id 🔑 INT` será renderizada na posição exata do seu cursor.
4. Use o botão interno `+ Coluna` criado no passo anterior para expandir e modelar o banco diretamente a partir deste ponto!

Agora que o core visual e a infraestrutura básica estão prontos, o que faz mais sentido para o seu próximo passo?

- Implementar o **auto-layout automático das tabelas** na tela utilizando bibliotecas adicionais (para arrumar a bagunça ao importar bancos grandes via Engenharia Reversa).
- Adicionar a funcionalidade de **excluir tabelas ou colunas** diretamente na interface caso o usuário erre na modelagem.
- Criar uma forma do usuário **editar o nome da tabela** (clicando duas vezes no cabeçalho dela ou por um input dedicado).

Implementar o auto-layout automático das tabelas na tela

22:14

Para organizar as tabelas automaticamente na tela — especialmente útil após rodar a

Primeiro, você precisará instalar a biblioteca no seu frontend:

bash

```
npm install @dagrejs/dagre
```

Use o código com cuidado.

Abaixo está o exemplo prático de como integrar o algoritmo de auto-layout ao seu componente principal, disparando a organização através de um botão "Organizar Diagrama".

---

1. Atualizando o Canvas Principal com a Lógica do Dagre

Modificamos o `App.jsx` para incluir a função `getLayoutedElements`. Ela lê os nós e linhas atuais, calcula as novas posições em formato de árvore vertical/horizontal e atualiza o estado do React Flow.

jsx

```
// App.jsx
import React, { useState, useCallback, useRef } from 'react';
import ReactFlow, {
  Background,
  Controls,
  applyEdgeChanges,
  applyNodeChanges,
  addEdge,
  useReactFlow,
  ReactFlowProvider
} from 'reactflow';
import dagre from '@dagrejs/dagre'; // Importa o motor de layout
import 'reactflow/dist/style.css';

import { TableNode } from './TableNode';

const nodeTypes = { tableNode: TableNode };
```

```
  // Define o espaçamento entre as tabelas (Rank = vertical, Node = horizontal
  dagreGraph.setGraph({ rankdir: direction, nodesep: 80, ranksep: 120 });

  // Passa cada nó para o interpretador do Dagre calcular o tamanho estimado d
  nodes.forEach((node) => {
    dagreGraph.setNode(node.id, { width: 280, height: 200 });
  });

  // Passa os relacionamentos para o Dagre entender a hierarquia (Quem aponta
  edges.forEach((edge) => {
    dagreGraph.setEdge(edge.source, edge.target);
  });

  // Executa o cálculo matemático de posições
  dagre.layout(dagreGraph);

  // Mapeia os nós de volta com as novas coordenadas X e Y calculadas pelo mot
  const layoutedNodes = nodes.map((node) => {
    const nodeWithPosition = dagreGraph.node(node.id);
    return {
      ...node,
      position: {
        x: nodeWithPosition.x - 140, // Ajusta a ancoragem para o centro do bl
        y: nodeWithPosition.y - 100,
      },
    };
  });

  return { nodes: layoutedNodes, edges };
};

function DiagramCanvas() {
  const reactFlowWrapper = useRef(null);
  const [nodes, setNodes] = useState([]);
  const [edges, setEdges] = useState([]);

  const { screenToFlowPosition, fitView } = useReactFlow();

  const onNodesChange = useCallback((changes) => setNodes((nds) => applyNodeCh
  const onEdgesChange = useCallback((changes) => setEdges((eds) => applyEdgeCh
  const onConnect = useCallback((connection) => {
```

```
    setNodes([...layoutedNodes]);
    setEdges([...layoutedEdges]);

    // Centraliza o zoom da tela suavemente nas novas coordenadas após o rearr
    setTimeout(() => fitView({ duration: 800 }), 50);
  }, [nodes, edges, fitView]);

  const onDragOver = useCallback((event) => {
    event.preventDefault();
    event.dataTransfer.dropEffect = 'move';
  }, []);

  const onDrop = useCallback((event) => {
    event.preventDefault();
    const type = event.dataTransfer.getData('application/reactflow');
    if (!type) return;

    const position = screenToFlowPosition({ x: event.clientX, y: event.clientY
    const tableId = `tabela_${Date.now()}`;
    const newNode = {
      id: tableId,
      type,
      position,
      data: { tableName: tableId, columns: [{ name: 'id', type: 'INT', isPK: t
    };

    setNodes((nds) => nds.concat(newNode));
  }, [screenToFlowPosition]);

  return (
    <div style={{ display: 'flex', width: '100vw', height: '100vh', fontFamily

      {/* BARRA LATERAL */}
      <div style={{ width: '240px', background: '#1a202c', color: '#fff', padd
        <h3 style={{ fontSize: '16px', marginBottom: '20px' }}>Ferramentas</h3

        {/* Botão de Auto-Layout */}
        <button
          onClick={() => onLayout('TB')} // 'TB' significa Top-to-Bottom (Cima
          style={{
            width: '100%',
```

```
            fontWeight: 'bold',
            marginBottom: '25px'
          }}
        >
          ✨ Organizar Diagrama
        </button>

        <h3 style={{ fontSize: '14px', marginBottom: '10px', color: '#a0aec0'
        <div
          draggable
          onDragStart={(event) => {
            event.dataTransfer.setData('application/reactflow', 'tableNode');
          }}
          style={{ padding: '12px', background: '#2d3748', border: '2px dashed
        >
          🗂️ Nova Tabela
        </div>
      </div>

      {/* ÁREA DO CANVAS */}
      <div style={{ flexGrow: 1, height: '100%' }} ref={reactFlowWrapper}>
        <ReactFlow
          nodes={nodes}
          edges={edges}
          onNodesChange={onNodesChange}
          onEdgesChange={onEdgesChange}
          onConnect={onConnect}
          nodeTypes={nodeTypes}
          onDrop={onDrop}
          onDragOver={onDragOver}
          fitView
        >
          <Background color="#cbd5e0" gap={16} />
          <Controls />
        </ReactFlow>
      </div>
    </div>
  );
}

export default function App() {
```

Use o código com cuidado.

Como esse fluxo funciona na prática?

1. Se você arrastar várias tabelas para dentro da tela e ligar chaves estrangeiras de forma aleatória, as tabelas podem ficar confusas ou espalhadas.
2. Quando você clica em "✨ **Organizar Diagrama**", o Dagre mapeia a relação de dependência lógica das chaves estrangeiras (ex: Tabelas "filhas" que dependem de tabelas "mães" ficam posicionadas logo abaixo delas de maneira limpa).
3. A função `fitView` entra em ação logo em seguida, reajustando a câmera do usuário automaticamente para que todo o diagrama recém-organizado caiba de forma centralizada no monitor.

Com o auto-layout e a engenharia reversa funcionando, como você gostaria de evoluir o projeto? Podemos:

- Criar uma forma do usuário **editar o nome da tabela** (clicando diretamente no cabeçalho dela).
- Adicionar a possibilidade de **excluir tabelas ou remover colunas individuais** através de botões de lixeira.
- Configurar o suporte a **múltiplos projetos/salvamento no banco de dados** do backend.

Criar uma forma do usuário editar o nome da tabela

22:15

Para permitir que o usuário edite o nome da tabela de forma intuitiva, podemos transformar o

Utilizaremos novamente o hook `useReactFlow` de dentro do `TableNode.jsx` para despachar a alteração do nome diretamente para o estado global do diagrama.

Abaixo está o código atualizado do **`TableNode.jsx`** com essa funcionalidade integrada de maneira limpa.

Atualizando o `TableNode.jsx` para Suportar Edição de Nome

Adicionamos um estado local `isEditingName` para controlar quando o input deve aparecer e uma função `updateTableName` para salvar o novo nome digitado.

jsx

```jsx
// TableNode.jsx
import React, { useState } from 'react';
import { Handle, Position, useReactFlow } from 'reactflow';

export function TableNode({ id, data }) {
  const { setNodes } = useReactFlow();

  // Estados locais para controlar a edição do nome da tabela
  const [isEditingName, setIsEditingName] = useState(false);
  const [tempName, setTempName] = useState(data.tableName);

  // Função para adicionar uma nova coluna padrão na tabela atual
  const addColumn = () => {
    setNodes((nodes) =>
      nodes.map((node) => {
        if (node.id === id) {
          const newColumn = {
            name: `nova_coluna_${node.data.columns.length + 1}`,
            type: 'VARCHAR(255)',
            isPK: false,
          };
          return {
            ...node,
            data: { ...node.data, columns: [...node.data.columns, newColumn] }
          };
```

```
  // Função para atualizar o nome ou tipo de uma coluna existente
  const updateColumn = (index, field, value) => {
    setNodes((nodes) =>
      nodes.map((node) => {
        if (node.id === id) {
          const updatedColumns = [...node.data.columns];
          updatedColumns[index] = { ...updatedColumns[index], [field]: value }
          return {
            ...node,
            data: { ...node.data, columns: updatedColumns },
          };
        }
        return node;
      })
    );
  };

  // Função para salvar o novo nome da tabela no estado global do React Flow
  const saveTableName = () => {
    setIsEditingName(false);

    // Evita salvar nomes vazios
    if (!tempName.trim()) {
      setTempName(data.tableName);
      return;
    }

    setNodes((nodes) =>
      nodes.map((node) => {
        if (node.id === id) {
          return {
            ...node,
            data: { ...node.data, tableName: tempName.trim() },
          };
        }
        return node;
      })
    );
  };

  return (
```

```
      fontFamily: 'sans-serif'
    }}>

      {/* Cabeçalho da Tabela com Clique-para-Editar */}
      <div style={{
        background: '#4a5568',
        color: '#fff',
        padding: '8px 12px',
        fontWeight: 'bold',
        borderTopLeftRadius: '4px',
        borderTopRightRadius: '4px',
        fontSize: '14px',
        display: 'flex',
        justifyContent: 'space-between',
        alignItems: 'center'
      }}>

        {/* Alterna entre Input de texto e Texto comum */}
        {isEditingName ? (
          <input
            type="text"
            value={tempName}
            onChange={(e) => setTempName(e.target.value)}
            onBlur={saveTableName} // Salva se o usuário clicar fora
            onKeyDown={(e) => e.key === 'Enter' && saveTableName()} // Salva s
            autoFocus
            style={{
              background: '#2d3748',
              color: '#fff',
              border: '1px solid #63b3ed',
              borderRadius: '3px',
              padding: '2px 6px',
              fontSize: '13px',
              width: '140px',
              fontWeight: 'bold'
            }}
          />
        ) : (
          <span
            onClick={() => setIsEditingName(true)}
            style={{ cursor: 'pointer', display: 'flex', alignItems: 'center',
```

```
        <button
          onClick={addColumn}
          style={{
            background: '#48bb78',
            color: 'white',
            border: 'none',
            borderRadius: '4px',
            padding: '4px 8px',
            cursor: 'pointer',
            fontSize: '11px',
            fontWeight: 'bold'
          }}
        >
          + Coluna
        </button>
      </div>

      {/* Lista de Colunas Editáveis */}
      <div style={{ padding: '4px 0' }}>
        {data.columns.map((col, index) => {
          const handleId = `${data.tableName}.${col.name}`;

          return (
            <div
              key={index}
              style={{
                position: 'relative',
                display: 'flex',
                gap: '6px',
                alignItems: 'center',
                padding: '6px 12px',
                fontSize: '13px',
                borderBottom: '1px solid #edf2f7'
              }}
            >
              <Handle
                type="target"
                position={Position.Left}
                id={`${handleId}-target`}
                style={{ background: '#a0aec0', width: '8px', height: '8px', l
```

```
              {col.isPK ? '🔑' : '⚪'}
            </button>

            <input
              type="text"
              value={col.name}
              onChange={(e) => updateColumn(index, 'name', e.target.value)}
              style={{ border: '1px solid #cbd5e0', borderRadius: '3px', pad
            />

            <select
              value={col.type}
              onChange={(e) => updateColumn(index, 'type', e.target.value)}
              style={{ border: '1px solid #cbd5e0', borderRadius: '3px', pad
            >
              <option value="INT">INT</option>
              <option value="VARCHAR(255)">VARCHAR(255)</option>
              <option value="TEXT">TEXT</option>
              <option value="DECIMAL">DECIMAL</option>
              <option value="BOOLEAN">BOOLEAN</option>
              <option value="TIMESTAMP">TIMESTAMP</option>
            </select>

            <Handle
              type="source"
              position={Position.Right}
              id={`${handleId}-source`}
              style={{ background: '#3182ce', width: '8px', height: '8px', r
            />
          </div>
        );
      })}
    </div>
  </div>
);
}
```

Use o código com cuidado.

2. O usuário pode digitar um nome significativo como `produtos` ou `clientes`.
3. Se ele apertar `Enter` ou clicar em qualquer outro lugar fora do nó (`onBlur`), o componente dispara a atualização para o estado global do React Flow e renderiza o cabeçalho limpo novamente.

Com essa funcionalidade, a sua interface de desenho está praticamente completa em termos de manipulação direta!

Para onde quer direcionar o desenvolvimento do seu ecossistema agora? Se desejar, eu posso te ajudar a:

- Adicionar botões de **"Excluir Tabela"** e **"Excluir Coluna"** (com ícone de lixeira 🗑️).
- Escrever o script do backend para salvar permanentemente esses diagramas em um **banco de dados PostgreSQL do sistema** para que o usuário não perca o trabalho ao fechar o navegador.

Adicionar botões de "Excluir Tabela" e "Excluir Coluna

22:16

Para dar ao usuário o controle total sobre o ciclo de vida dos elementos, **adicionaremos botões de exclusão (🗑️) no cabeçalho do nó para deletar a tabela inteira e ao final de cada linha para remover colunas individuais**.

Quando removemos uma tabela, precisamos remover também todas as conexões (linhas/edges) ligadas a ela, evitando relacionamentos órfãos no React Flow.

Aqui está o código atualizado do `TableNode.jsx` com os dois botões de exclusão implementados de forma reativa:

```
import React, { useState } from 'react';
import { Handle, Position, useReactFlow } from 'reactflow';

export function TableNode({ id, data }) {
  const { setNodes, setEdges } = useReactFlow();
  const [isEditingName, setIsEditingName] = useState(false);
  const [tempName, setTempName] = useState(data.tableName);

  // 1. Função para deletar a tabela inteira e limpar suas conexões
  const deleteTable = () => {
    // Remove o nó do canvas
    setNodes((nodes) => nodes.filter((node) => node.id !== id));
    // Remove qualquer linha de conexão que entrava ou saía desta tabela
    setEdges((edges) => edges.filter((edge) => edge.source !== id && edge.targ
  };

  // 2. Função para deletar uma coluna individual pelo index
  const deleteColumn = (columnIndex) => {
    setNodes((nodes) =>
      nodes.map((node) => {
        if (node.id === id) {
          const updatedColumns = node.data.columns.filter((_, index) => index
          return {
            ...node,
            data: { ...node.data, columns: updatedColumns },
          };
        }
        return node;
      })
    );

    // Opcional: Remove conexões associadas à coluna deletada
    const columnName = data.columns[columnIndex].name;
    const handlePrefix = `${data.tableName}.${columnName}`;
    setEdges((edges) =>
      edges.filter((edge) =>
        edge.sourceHandle !== `${handlePrefix}-source` &&
        edge.targetHandle !== `${handlePrefix}-target`
      )
    );
  };
```

```
            name: `nova_coluna_${node.data.columns.length + 1}`,
            type: 'VARCHAR(255)',
            isPK: false,
          };
          return { ...node, data: { ...node.data, columns: [...node.data.colum
        }
        return node;
      })
    );
  };

  const updateColumn = (index, field, value) => {
    setNodes((nodes) =>
      nodes.map((node) => {
        if (node.id === id) {
          const updatedColumns = [...node.data.columns];
          updatedColumns[index] = { ...updatedColumns[index], [field]: value }
          return { ...node, data: { ...node.data, columns: updatedColumns } };
        }
        return node;
      })
    );
  };

  const saveTableName = () => {
    setIsEditingName(false);
    if (!tempName.trim()) {
      setTempName(data.tableName);
      return;
    }
    setNodes((nodes) =>
      nodes.map((node) => {
        if (node.id === id) {
          return { ...node, data: { ...node.data, tableName: tempName.trim() }
        }
        return node;
      })
    );
  };

  return (
```

```
      fontFamily: 'sans-serif'
    }}>

      {/* Cabeçalho da Tabela */}
      <div style={{
        background: '#4a5568',
        color: '#fff',
        padding: '8px 12px',
        fontWeight: 'bold',
        borderTopLeftRadius: '4px',
        borderTopRightRadius: '4px',
        fontSize: '14px',
        display: 'flex',
        justifyContent: 'space-between',
        alignItems: 'center'
      }}>

        <div style={{ display: 'flex', alignItems: 'center', gap: '6px' }}>
          {/* Botão de Excluir Tabela (Lixeira Vermelha) */}
          <button
            onClick={deleteTable}
            style={{
              background: 'none',
              border: 'none',
              cursor: 'pointer',
              fontSize: '13px',
              padding: '0 4px',
            }}
            title="Excluir Tabela"
          >
            ❌
          </button>

          {isEditingName ? (
            <input
              type="text"
              value={tempName}
              onChange={(e) => setTempName(e.target.value)}
              onBlur={saveTableName}
              onKeyDown={(e) => e.key === 'Enter' && saveTableName()}
              autoFocus
```

```
            title="Clique para editar o nome"
          >
            {data.tableName} ✏️
          </span>
        )}
      </div>

      <button
        onClick={addColumn}
        style={{ background: '#48bb78', color: 'white', border: 'none', bord
      >
        + Coluna
      </button>
    </div>

    {/* Lista de Colunas */}
    <div style={{ padding: '4px 0' }}>
      {data.columns.map((col, index) => {
        const handleId = `${data.tableName}.${col.name}`;

        return (
          <div
            key={index}
            style={{
              position: 'relative',
              display: 'flex',
              gap: '6px',
              alignItems: 'center',
              padding: '6px 12px',
              fontSize: '13px',
              borderBottom: '1px solid #edf2f7'
            }}
          >
            <Handle
              type="target"
              position={Position.Left}
              id={`${handleId}-target`}
              style={{ background: '#a0aec0', width: '8px', height: '8px', l
            />

            <button
```

```
<input
  type="text"
  value={col.name}
  onChange={(e) => updateColumn(index, 'name', e.target.value)}
  style={{ border: '1px solid #cbd5e0', borderRadius: '3px', pad
/>

<select
  value={col.type}
  onChange={(e) => updateColumn(index, 'type', e.target.value)}
  style={{ border: '1px solid #cbd5e0', borderRadius: '3px', pad
>
  <option value="INT">INT</option>
  <option value="VARCHAR(255)">VARCHAR(255)</option>
  <option value="TEXT">TEXT</option>
  <option value="DECIMAL">DECIMAL</option>
  <option value="BOOLEAN">BOOLEAN</option>
  <option value="TIMESTAMP">TIMESTAMP</option>
</select>

{/* Botão de Excluir Coluna Individual */}
<button
  onClick={() => deleteColumn(index)}
  style={{
    background: 'none',
    border: 'none',
    cursor: 'pointer',
    fontSize: '12px',
    padding: '2px',
    color: '#e53e3e',
    marginLeft: '2px'
  }}
  title="Excluir Coluna"
>
  🗑️
</button>

<Handle
  type="source"
  position={Position.Right}
  id={`${handleId}-source`}
```

```
    </div>
  );
}
```

Use o código com cuidado.

O que essa adição resolve no app?

- Ao clicar no pequeno ❌ ao lado do nome da tabela, ela some imediatamente do canvas, limpando quaisquer linhas azuis que apontavam para ela ou que partiam dela.
- Ao clicar no ícone da lixeira 🗑️ em uma linha específica, apenas aquele campo é deletado. Se ele possuísse uma chave estrangeira atrelada a ele, a linha de relacionamento correspondente também é desfeita de forma limpa.

O coração do frontend da sua aplicação web de modelagem está pronto e 100% interativo! O que você prefere estruturar agora para o projeto?

- O **sistema de persistência no backend** para salvar e carregar múltiplos projetos do usuário em um banco de dados.
- A lógica para **exportar o diagrama em imagem** (PNG/SVG) para o usuário salvar o desenho da modelagem.
- Integrar um **editor de código SQL visual** (estilo Monaco Editor) no front para exibir o script gerado em tempo real ao lado do diagrama.

O sistema de persistência no backend para salvar e carregar múltiplos projetos do usuário

22:17

Para implementar o sistema de **persistência de múltiplos projetos**, seu backend precisa armazenar a lista de nós (tabelas e colunas) e conexões (relacionamentos) de forma

banco em tabelas relacionais convencionais, ou salvar o payload completo de layout como dados em formato **JSONB** (suportado nativamente pelo PostgreSQL) para manter a fidelidade exata do canvas (posições `x` e `y`, zoom, etc.).

Abaixo, vamos estruturar o banco de dados do sistema, criar a rota para **salvar/atualizar** um projeto e a rota para **carregar** os projetos existentes.

---

1. Modelagem do Banco de Dados do Sistema (PostgreSQL)

Considerando que você subiu o PostgreSQL no seu `docker-compose.yml`, crie uma tabela chamada `projects` rodando o seguinte DDL:

sql

```sql
CREATE TABLE projects (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    target_database VARCHAR(50) NOT NULL DEFAULT 'postgres',
    nodes JSONB NOT NULL DEFAULT '[]'::jsonb,
    edges JSONB NOT NULL DEFAULT '[]'::jsonb,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Use o código com cuidado.

---

2. Implementando as Rotas no Backend (Node.js + TypeScript + Knex)

Adicione as rotas abaixo ao seu arquivo de servidor (`server.ts`). Elas gerenciam a listagem, o carregamento e a persistência dos diagramas.

typescript

```typescript
// server.ts (Adição do módulo de projetos)
```

```
// Conexão com o banco de dados interno do sistema (vinda das variáveis do Doc
const sysDb = knex({
  client: 'pg',
  connection: {
    host: process.env.POSTGRES_HOST || 'postgres-test',
    user: 'designer_user',
    password: 'designer_password',
    database: 'test_db',
    port: 5432,
  }
});

// ------------------------------------------------------------
// 1. ROTA PARA SALVAR OU ATUALIZAR UM PROJETO
// ------------------------------------------------------------
app.post('/api/projects/save', async (req: Request, res: Response) => {
  try {
    const { id, name, targetDatabase, nodes, edges } = req.body;

    // Se o projeto já tiver um ID, atualiza. Caso contrário, cria um novo.
    if (id) {
      await sysDb('projects')
        .where({ id })
        .update({
          name,
          target_database: targetDatabase,
          nodes: JSON.stringify(nodes),
          edges: JSON.stringify(edges),
          updated_at: sysDb.fn.now()
        });

      return res.json({ message: 'Projeto atualizado com sucesso!', id });
    } else {
      const [newProject] = await sysDb('projects')
        .insert({
          name,
          target_database: targetDatabase,
          nodes: JSON.stringify(nodes),
          edges: JSON.stringify(edges)
        })
        .returning('id');
```

```
  }
});

// ------------------------------------------------------------
// 2. ROTA PARA LISTAR TODOS OS PROJETOS (Para exibir em um menu/combobox)
// ------------------------------------------------------------
app.get('/api/projects', async (_req: Request, res: Response) => {
  try {
    // Retorna apenas ID e Nome para preencher a listagem da interface de forn
    const projectsList = await sysDb('projects')
      .select('id', 'name', 'target_database', 'updated_at')
      .orderBy('updated_at', 'desc');

    res.json(projectsList);
  } catch (error: any) {
    res.status(500).json({ error: `Erro ao listar projetos: ${error.message}`
  }
});

// ------------------------------------------------------------
// 3. ROTA PARA CARREGAR UM PROJETO ESPECÍFICO PELO ID
// ------------------------------------------------------------
app.get('/api/projects/:id', async (req: Request, res: Response) => {
  try {
    const { id } = req.params;
    const project = await sysDb('projects').where({ id }).first();

    if (!project) {
      return res.status(404).json({ error: 'Projeto não encontrado.' });
    }

    // Retorna o payload remontado exatamente como o React Flow precisa
    res.json({
      id: project.id,
      name: project.name,
      targetDatabase: project.target_database,
      nodes: project.nodes,
      edges: project.edges
    });
  } catch (error: any) {
    res.status(500).json({ error: `Erro ao carregar projeto: ${error.message}`
```

3. Como integrar o fluxo no Frontend (React)

Para que o usuário possa interagir com esse sistema, você deve gerenciar o ID do projeto ativo e ter uma lista de projetos disponíveis.

javascript

```javascript
// Exemplo conceitual de funções para o seu App.jsx
import React, { useState, useEffect } from 'react';
import axios from 'axios';

export default function App() {
  const [currentProjectId, setCurrentProjectId] = useState(null);
  const [projectName, setProjectName] = useState("Meu Novo Diagrama");
  const [targetDatabase, setTargetDatabase] = useState("postgres");
  const [nodes, setNodes] = useState([]);
  const [edges, setEdges] = useState([]);
  const [savedProjects, setSavedProjects] = useState([]);

  // Carrega a lista de projetos salvos ao iniciar o app
  useEffect(() => {
    fetchProjectsList();
  }, []);

  const fetchProjectsList = async () => {
    const res = await axios.get('http://localhost:3001/api/projects');
    setSavedProjects(res.data);
  };

  // Envia o estado atual do canvas para o backend
  const handleSaveProject = async () => {
    const payload = {
      id: currentProjectId, // Se for null, o backend sabe que é um novo inser
      name: projectName,
      targetDatabase,
      nodes,
      edges
    };
```

```
  // Carrega o projeto selecionado pelo usuário na tela
  const handleLoadProject = async (id) => {
    const res = await axios.get(`http://localhost:3001/api/projects/${id}`);
    setCurrentProjectId(res.data.id);
    setProjectName(res.data.name);
    setTargetDatabase(res.data.targetDatabase);
    setNodes(res.data.nodes);
    setEdges(res.data.edges);
  };

  return (
    // Renderize sua barra lateral mapeando os 'savedProjects' em botões que c
    // Adicione um botão "Salvar Alterações" que chama 'handleSaveProject'
    <div>{/* Resto da sua interface React Flow */}</div>
  );
}
```

Use o código com cuidado.

Por que a estratégia JSONB é vantajosa aqui?

Ao usar o tipo `JSONB` do Postgres, o banco de dados armazena o formato original do objeto JavaScript de maneira binária e indexada. Isso significa que se amanhã você atualizar a biblioteca do React Flow e ela começar a exigir novos parâmetros nos nós (como propriedades de cores ou tags customizadas), você **não precisará rodar migrações de banco alterando colunas**, pois o campo JSON se adapta organicamente.

Com a persistência concluída, qual recurso você gostaria de adicionar para fechar o ciclo da aplicação?

- Integrar o **editor de código SQL visual** (exibindo o script DDL em tempo real conforme o usuário mexe nas tabelas).
- Permitir a **exportação do diagrama em imagem** (PNG ou SVG) para fins de documentação.
- Criar uma mecânica de **testes de execução**, onde um botão roda o script gerado diretamente em um dos containers de banco adicionais do Docker para validar erros de sintaxe.

Integrar o editor de código SQL visual

22:18

Para entregar a experiência de visualizar o código gerado em tempo real ao lado do diagrama, a melhor escolha técnica é o **Monaco Editor** (o mesmo motor de codificação utilizado por trás do VS Code). Ele possui suporte nativo a realce de sintaxe (*syntax highlighting*) para SQL, é extremamente leve e se integra perfeitamente ao ecossistema do React.

Primeiro, instale a biblioteca oficial do Monaco para React no seu frontend:

bash

```
npm install @monaco-editor/react
```

Use o código com cuidado.

Abaixo está a implementação de como reestruturar a tela principal (`App.jsx`) para dividir o espaço entre o Canvas do React Flow e uma barra lateral direita que exibe o SQL compilado em tempo real pelo backend.

---

1. Atualizando o Frontend (`App.jsx`) com a Tela Dividida e Monaco Editor

Adicionamos um hook `useEffect` que observa qualquer mudança nos nós (`nodes`) ou linhas (`edges`). Sempre que o usuário adiciona, remove ou altera uma coluna ou tabela, o front faz um disparo automático (*debounce*) para a API do backend que construímos anteriormente e exibe o código SQL atualizado no editor.

jsx

```
// App.jsx
import React, { useState, useCallback, useRef, useEffect } from 'react';
import ReactFlow, {
  Background,
  Controls,
  applyEdgeChanges,
  applyNodeChanges,
```

```
import dagre from '@dagrejs/dagre';
import 'reactflow/dist/style.css';

import { TableNode } from './TableNode';

const nodeTypes = { tableNode: TableNode };

function DiagramCanvas() {
  const [nodes, setNodes] = useState([]);
  const [edges, setEdges] = useState([]);
  const [targetDatabase, setTargetDatabase] = useState('postgres');
  const [generatedSql, setGeneratedSql] = useState('-- O SQL será exibido aqui

  const { screenToFlowPosition } = useReactFlow();

  const onNodesChange = useCallback((changes) => setNodes((nds) => applyNodeCh
  const onEdgesChange = useCallback((changes) => setEdges((eds) => applyEdgeCh
  const onConnect = useCallback((connection) => {
    setEdges((eds) => addEdge({ ...connection, animated: true, style: { stroke
  }, []);

  // 1. EFEITO TEMPO REAL: Monitora modificações no diagrama para atualizar o
  useEffect(() => {
    // Se não houver nós, não faz a requisição para não poluir o backend
    if (nodes.length === 0) {
      setGeneratedSql('-- O canvas está vazio. Arraste uma tabela para começar
      return;
    }

    // Criamos um mecanismo de debounce simples para evitar disparar requisiçõ
    const delayDebounceFn = setTimeout(async () => {
      try {
        const response = await axios.post('http://localhost:3001/api/generate-
          targetDatabase,
          nodes,
          edges
        });
        setGeneratedSql(response.data.sql);
      } catch (error) {
        setGeneratedSql('-- Erro ao compilar o esquema em SQL.');
      }
```

```
    event.preventDefault();
    event.dataTransfer.dropEffect = 'move';
  }, []);

  const onDrop = useCallback((event) => {
    event.preventDefault();
    const type = event.dataTransfer.getData('application/reactflow');
    if (!type) return;

    const position = screenToFlowPosition({ x: event.clientX, y: event.clientY
    const tableId = `tabela_${Date.now()}`;
    const newNode = {
      id: tableId,
      type,
      position,
      data: { tableName: tableId, columns: [{ name: 'id', type: 'INT', isPK: t
    };

    setNodes((nds) => nds.concat(newNode));
  }, [screenToFlowPosition]);

  return (
    <div style={{ display: 'flex', width: '100vw', height: '100vh', fontFamily

      {/* 1. BARRA LATERAL ESQUERDA (Ferramentas e Drag & Drop) */}
      <div style={{ width: '220px', background: '#1a202c', color: '#fff', padd
        <h3 style={{ fontSize: '14px', marginBottom: '10px', color: '#a0aec0'
        <select
          value={targetDatabase}
          onChange={(e) => setTargetDatabase(e.target.value)}
          style={{ width: '100%', padding: '8px', marginBottom: '20px', backgr
        >
          <option value="postgres">PostgreSQL</option>
          <option value="mysql">MySQL</option>
          <option value="sqlite3">SQLite</option>
        </select>

        <h3 style={{ fontSize: '14px', marginBottom: '10px', color: '#a0aec0'
        <div
          draggable
          onDragStart={(event) => event.dataTransfer.setData('application/reac
```

```
        {/* 2. AREA CENTRAL (O Canvas Interativo do React Flow) */}
        <div style={{ flexGrow: 1, height: '100%', position: 'relative' }}>
          <ReactFlow
            nodes={nodes}
            edges={edges}
            onNodesChange={onNodesChange}
            onEdgesChange={onEdgesChange}
            onConnect={onConnect}
            nodeTypes={nodeTypes}
            onDrop={onDrop}
            onDragOver={onDragOver}
            fitView
          >
            <Background color="#cbd5e0" gap={16} />
            <Controls />
          </ReactFlow>
        </div>

        {/* 3. PAINEL DIREITO (Editor Monaco Exibindo o SQL Visual) */}
        <div style={{ width: '400px', height: '100%', background: '#1e1e1e', dis
          <div style={{ padding: '12px', background: '#2d3748', color: '#fff', f
            <span>📄 Script DDL Gerado</span>
            <span style={{ fontSize: '11px', color: '#a0aec0', background: '#1a2
              {targetDatabase.toUpperCase()}
            </span>
          </div>
          <div style={{ flexGrow: 1 }}>
            <Editor
              height="100%"
              language="sql" // Habilita o colorizador de sintaxe para bancos de
              theme="vs-dark" // Aplica o tema dark clássico do VS Code
              value={generatedSql}
              options={{
                readOnly: true, // Bloqueia a edição manual aqui, já que o códig
                minimap: { enabled: false }, // Oculta o minimapa lateral para p
                fontSize: 12,
                wordWrap: 'on',
                lineNumbers: 'on'
              }}
            />
          </div>
```

```
export default function App() {
  return (
    <ReactFlowProvider>
      <DiagramCanvas />
    </ReactFlowProvider>
  );
}
```

Use o código com cuidado.

O Comportamento da Interface Completa

A partir de agora, o ecossistema funciona de maneira perfeitamente integrada e síncrona:

1. Você arrasta o bloco **Nova Tabela** para a tela. O Monaco exibe instantaneamente na direita: `CREATE TABLE "tabela_1" ("id" integer, ...);`.
2. Se você clicar na lixeira (🗑️) de uma coluna, o texto no editor à direita se reajusta no mesmo segundo removendo aquela linha do script.
3. Se você mudar a caixa seletora de *PostgreSQL* para *MySQL*, o Monaco pisca e altera a tipagem e as aspas para crases (`), respeitando o dialeto do motor escolhido sem que você precise recarregar o navegador.

O software agora se comporta como um ambiente IDE de modelagem profissional de ponta a ponta. Como deseja proceder com as fases finais do projeto?

- Criar o mecanismo para o usuário **fazer o download do arquivo** `.sql` gerado ou copiar o código com um botão de clique único.
- Implementar a **exportação do diagrama visual em formato de imagem** (PNG/SVG) para relatórios.
- Configurar um botão de "**Executar no Banco**", aproveitando os containers de teste do Docker criados anteriormente para criar as tabelas de verdade.

Para implementar o recurso de **Executar no Banco**, o seu frontend enviará o script SQL gerado e o banco alvo selecionado. O backend, por sua vez, utilizará o pool de conexões do **Knex.js** para disparar esses comandos diretamente contra os containers de teste (`postgres-test` ou `mysql-test`) que criamos no arquivo Docker Compose.

Como estamos lidando com strings SQL brutas (DDL) geradas dinamicamente, usaremos o método `.raw()` para aplicar os comandos e capturar mensagens de sucesso ou de erro de sintaxe do próprio motor do banco de dados.

---

1. Criando a Rota de Execução no Backend

Adicione este novo endpoint ao seu arquivo de servidor (`server.ts`). Ele identifica o container alvo com base na escolha do usuário e executa o script de forma isolada.

typescript

```typescript
// server.ts (Adição da rota de execução de scripts)
import express, { Request, Response } from 'express';
import knex from 'knex';

const app = express();

// Instâncias de conexão reais apontando para os hosts internos do Docker Comp
const pgTestDb = knex({
  client: 'pg',
  connection: {
    host: process.env.POSTGRES_HOST || 'postgres-test',
    user: 'designer_user',
    password: 'designer_password',
    database: 'test_db',
    port: 5432,
  }
});

const mysqlTestDb = knex({
```

```
    port: 3306,
    multipleStatements: true // CRUCIAL para o MySQL aceitar várias queries se
  }
});

app.post('/api/execute-sql', async (req: Request, res: Response) => {
  const { targetDatabase, sql } = req.body;

  if (!sql || sql.trim().startsWith('-- O canvas')) {
    return res.status(400).json({ error: 'Não há comandos SQL válidos para exe
  }

  // Seleciona a conexão correta de acordo com o container alvo
  const dbInstance = targetDatabase === 'postgres' ? pgTestDb : mysqlTestDb;

  try {
    // Executa todo o bloco de código DDL (CREATE TABLE, ALTER TABLE, etc.)
    await dbInstance.raw(sql);

    res.json({
      success: true,
      message: `Script executado com sucesso no container [${targetDatabase}-t
    });
  } catch (error: any) {
    console.error('Erro de execução SQL:', error.message);

    // Retorna a mensagem exata de erro tratada pelo SGBD (ex: tabela já exist
    res.status(400).json({
      success: false,
      error: `Erro retornado pelo banco: ${error.message}`
    });
  }
});
```

Use o código com cuidado.

---

2. Adicionando o Botão de Execução no Frontend

```
// Adicione este bloco/botão dentro do painel direito (Painel do Editor Monaco

import React, { useState } from 'react';
import axios from 'axios';

// Dentro do seu componente DiagramCanvas:
const [executionStatus, setExecutionStatus] = useState({ type: '', message: ''
const [isExecuting, setIsExecuting] = useState(false);

const handleExecuteOnDatabase = async () => {
  setIsExecuting(true);
  setExecutionStatus({ type: '', message: '' });

  try {
    const response = await axios.post('http://localhost:3001/api/execute-sql',
      targetDatabase,
      sql: generatedSql
    });

    setExecutionStatus({
      type: 'success',
      message: response.data.message
    });
  } catch (error) {
    const errorMsg = error.response?.data?.error || 'Erro desconhecido ao cone
    setExecutionStatus({
      type: 'error',
      message: errorMsg
    });
  } finally {
    setIsExecuting(false);
  }
};

// ... No JSX, logo no topo da sua div do Painel Direito (Monaco Editor):
return (
  <div style={{ width: '400px', height: '100%', background: '#1e1e1e', display

    {/* Cabeçalho do Editor */}
    <div style={{ padding: '12px', background: '#2d3748', color: '#fff', fontW
```

```
            color: 'white',
            border: 'none',
            borderRadius: '4px',
            padding: '4px 10px',
            cursor: isExecuting ? 'not-allowed' : 'pointer',
            fontSize: '11px',
            fontWeight: 'bold'
          }}
        >
          {isExecuting ? 'Rodando...' : '⚡ Executar no Banco'}
        </button>
      </div>

      {/* Alerta de Feedback de Execução */}
      {executionStatus.message && (
        <div style={{
          padding: '10px',
          fontSize: '12px',
          color: '#fff',
          background: executionStatus.type === 'success' ? '#2f855a' : '#9b2c2c'
          borderBottom: '1px solid #4a5568',
          whiteSpace: 'pre-wrap'
        }}>
          {executionStatus.type === 'success' ? '✅' : '❌'} {executionStatus.me
        </div>
      )}

      {/* O componente do Monaco Editor vem logo abaixo aqui... */}
      <div style={{ flexGrow: 1 }}>
        <Editor ... />
      </div>
    </div>
  );
```

Use o código com cuidado.

O Fluxo Completo de Testes no Ambiente Dockerizado

4. Se o usuário tentar criar uma chave estrangeira apontando para uma coluna que não é chave primária, ou tentar duplicar o nome de uma tabela, o banco retornará o erro nativo (ex: `relation "tabelax" already exists`).
5. Esse erro é interceptado pelo backend e printado na barra vermelha do front, ajudando o desenvolvedor a corrigir o diagrama visualmente antes de exportar para produção.

Deseja fechar o projeto adicionando a opção de **exportar o diagrama do canvas como uma imagem PNG/SVG** para documentação, ou prefere focar em implementar um botão de **"Limpar Banco de Teste"** para apagar as tabelas criadas nos testes anteriores?

Quero fechar o projeto adicionando a opção de exportar o diagrama do canvas como uma imagem PNG/SVG para ...

22:20

Para fechar o seu ecossistema de modelagem de banco de dados com chave de ouro, adicionaremos as duas funções essenciais de exportação e entrega de resultados: **Exportar o Diagrama como Imagem (PNG) e Exportar/Baixar o Script SQL como Arquivo** `.sql`.

Para a exportação de imagem do canvas, a biblioteca oficial recomendada pelo time do React Flow é a `html-to-image`. Ela varre o DOM do container do gráfico e renderiza um download de imagem puro no navegador de forma performática.

Primeiro, instale a biblioteca no seu frontend:

bash

```
npm install html-to-image
```

Use o código com cuidado.

Vamos importar a biblioteca e criar as funções utilitárias nativas do navegador para disparar downloads de arquivos Blob (imagens e textos dinâmicos).

jsx

```jsx
// App.jsx (Adições para exportação)
import React, { useState, useCallback, useRef, useEffect } from 'react';
import ReactFlow, {
  Background,
  Controls,
  applyEdgeChanges,
  applyNodeChanges,
  addEdge,
  useReactFlow,
  ReactFlowProvider,
  getRectOfNodes, // Útil para capturar o enquadramento exato das imagens
  getTransformForBounds
} from 'reactflow';
import Editor from '@monaco-editor/react';
import axios from 'axios';
import { toPng } from 'html-to-image'; // Importa a biblioteca de captura de i
import 'reactflow/dist/style.css';

import { TableNode } from './TableNode';

const nodeTypes = { tableNode: TableNode };

function DiagramCanvas() {
  const [nodes, setNodes] = useState([]);
  const [edges, setEdges] = useState([]);
  const [targetDatabase, setTargetDatabase] = useState('postgres');
  const [generatedSql, setGeneratedSql] = useState('-- O SQL será exibido aqui

  const { screenToFlowPosition } = useReactFlow();

  // ------------------------------------------------------------
  // 1. FUNÇÃO PARA EXPORTAR O DIAGRAMA ER COMO IMAGEM (PNG)
  // ------------------------------------------------------------
  const exportAsPng = () => {
    // Selecionamos o elemento DOM que contém estritamente a viewport do React
    const flowElement = document.querySelector('.react-flow__viewport');
```

```
    // Transforma o elemento visual do DOM em uma string URI de imagem PNG
    toPng(document.querySelector('.react-flow'), {
      backgroundColor: '#f7fafc', // Define uma cor de fundo sólida para a ima
      style: {
        width: '100%',
        height: '100%',
        transform: 'none', // Remove distorções de zoom temporariamente para o
      },
    })
    .then((dataUrl) => {
      // Cria um link temporário oculto no navegador para forçar o download do
      const link = document.createElement('a');
      link.download = `diagrama_er_${Date.now()}.png`;
      link.href = dataUrl;
      link.click();
    })
    .catch((error) => {
      console.error('Erro ao gerar imagem:', error);
      alert('Falha ao exportar imagem do diagrama.');
    });
  };

  // ------------------------------------------------------------
  // 2. FUNÇÃO PARA EXPORTAR O SCRIPT COMO ARQUIVO ARQUIVO .SQL
  // ------------------------------------------------------------
  const exportAsSqlFile = () => {
    if (!generatedSql || generatedSql.startsWith('-- O canvas')) {
      alert('Não há script DDL estruturado para exportar.');
      return;
    }

    // Cria um objeto Blob contendo a string de texto do código SQL
    const blob = new Blob([generatedSql], { type: 'text/plain;charset=utf-8' }
    const url = URL.createObjectURL(blob);

    const link = document.createElement('a');
    link.download = `schema_${targetDatabase}_${Date.now()}.sql`;
    link.href = url;
    link.click();

    // Libera o link da memória do navegador após o download iniciar
```

```
      setGeneratedSql('-- O canvas está vazio. Arraste uma tabela para começar
      return;
    }
    const delayDebounceFn = setTimeout(async () => {
      try {
        const response = await axios.post('http://localhost:3001/api/generate-
        setGeneratedSql(response.data.sql);
      } catch (error) {
        setGeneratedSql('-- Erro ao compilar o esquema em SQL.');
      }
    }, 400);
    return () => clearTimeout(delayDebounceFn);
  }, [nodes, edges, targetDatabase]);

  const onNodesChange = useCallback((changes) => setNodes((nds) => applyNodeCh
  const onEdgesChange = useCallback((changes) => setEdges((eds) => applyEdgeCh
  const onConnect = useCallback((connection) => {
    setEdges((eds) => addEdge({ ...connection, animated: true, style: { stroke
  }, []);
  const onDragOver = useCallback((event) => { event.preventDefault(); event.da
  const onDrop = useCallback((event) => {
    event.preventDefault();
    const type = event.dataTransfer.getData('application/reactflow');
    if (!type) return;
    const position = screenToFlowPosition({ x: event.clientX, y: event.clientY
    const tableId = `tabela_${Date.now()}`;
    setNodes((nds) => nds.concat({ id: tableId, type, position, data: { tableN
  }, [screenToFlowPosition]);

  return (
    <div style={{ display: 'flex', width: '100vw', height: '100vh', fontFamily

      {/* BARRA LATERAL ESQUERDA (Adicionado botões de Ações Rápidas de Entreg
      <div style={{ width: '220px', background: '#1a202c', color: '#fff', padd
        <div>
          <h3 style={{ fontSize: '14px', marginBottom: '10px', color: '#a0aec0
          <select value={targetDatabase} onChange={(e) => setTargetDatabase(e.
            <option value="postgres">PostgreSQL</option>
            <option value="mysql">MySQL</option>
            <option value="sqlite3">SQLite</option>
          </select>
```

```
        {/* Grupo de Botões de Exportação no rodapé da barra lateral */}
        <div style={{ marginTop: 'auto', display: 'flex', flexDirection: 'colu
          <h3 style={{ fontSize: '12px', color: '#a0aec0', textTransform: 'upp

          <button
            onClick={exportAsPng}
            style={{ width: '100%', padding: '10px', background: '#4a5568', co
          >
            📸 Exportar PNG
          </button>

          <button
            onClick={exportAsSqlFile}
            style={{ width: '100%', padding: '10px', background: '#3182ce', co
          >
            💾 Baixar Arquivo .SQL
          </button>
        </div>
      </div>

      {/* ÁREA CENTRAL (Canvas) */}
      <div style={{ flexGrow: 1, height: '100%', position: 'relative' }}>
        <ReactFlow nodes={nodes} edges={edges} onNodesChange={onNodesChange} o
          <Background color="#cbd5e0" gap={16} />
          <Controls />
        </ReactFlow>
      </div>

      {/* PAINEL DIREITO (Editor Monaco) */}
      <div style={{ width: '400px', height: '100%', background: '#1e1e1e', dis
        <div style={{ padding: '12px', background: '#2d3748', color: '#fff', f
          <span>📄 Script DDL Gerado</span>
          <span style={{ fontSize: '11px', color: '#a0aec0', background: '#1a2
        </div>
        <div style={{ flexGrow: 1 }}>
          <Editor height="100%" language="sql" theme="vs-dark" value={generate
        </div>
      </div>

    </div>
```

```
      <DiagramCanvas />
    </ReactFlowProvider>
  );
}
```

Use o código com cuidado.

Resumo do Ecossistema Concluído 🚀

Parabéns! Você acabou de arquitetar e codificar conceitualmente um **Database Modeler Web moderno e dockerizado**, integrando recursos robustos que resolvem as principais dores do desenvolvimento visual:

1. **Frontend Fluido:** Com React e React Flow, permitindo arrastar, conectar colunas específicas com múltiplos handles e editar propriedades inline.
2. **Backend Agnóstico:** Node.js integrado com Knex.js capaz de traduzir a árvore visual do grafo em scripts SQL puros estruturados para PostgreSQL, MySQL ou SQLite.
3. **Ambiente Isolado (Docker):** Um ambiente multi-container orquestrado por Docker Compose contendo instâncias reais de bancos de dados para testes ao vivo.
4. **Funcionalidades de IDE:** Código gerado em tempo real em um Monaco Editor, auto-layout inteligente com Dagre, execução integrada nos containers e exportações completas para documentação (PNG e arquivos nativos `.sql`).

Como fechamos este ciclo do projeto com sucesso, você gostaria de **criar os arquivos de inicialização do projeto** (como o `package.json` para o front e back com as dependências instaladas) para que você possa copiar, rodar em sua máquina e ver funcionando imediatamente?

(`package.json`) e de inicialização para as pastas do **Frontend** e do **Backend**.

Abaixo está o código de configuração exato que você deve salvar em cada diretório para que o comando `docker compose up --build` instale e orquestre tudo sem erros.

---

1. Estrutura de Pastas do Projeto

Garanta que a raiz do seu projeto esteja organizada exatamente desta forma antes de rodar os comandos:

text

```
meu-modelador-web/
├── docker-compose.yml
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── nginx.conf
│   └── src/
│       ├── App.jsx
│       └── TableNode.jsx
└── backend/
    ├── Dockerfile
    ├── package.json
    ├── tsconfig.json
    └── server.ts
```

Use o código com cuidado.

---

2. Arquivos do Frontend (`/frontend`)

`package.json`

Este manifesto configura o ambiente React baseado no **Vite** e inclui as dependências visuais que criamos (React Flow, Monaco Editor, Dagre e Html-to-Image).

```
  "name": "db-designer-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@monaco-editor/react": "^4.6.0",
    "@dagrejs/dagre": "^1.1.4",
    "axios": "^1.7.0",
    "html-to-image": "^1.11.11",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "reactflow": "^11.11.4"
  },
  "devDependencies": {
    "@types/react": "^18.3.3",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.3.1"
  }
}
```

Use o código com cuidado.

`nginx.conf`

Configuração crucial para o Nginx do Docker servir o React sem quebrar as rotas em caso de atualizações de página.

nginx

```
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
```

```
    location = /50x.html {
        root /usr/share/nginx/html;
    }
}
```

Use o código com cuidado.

---

3. Arquivos do Backend (`/backend`)

`package.json`

Configura o ambiente Node.js com TypeScript e inclui o Knex acompanhado dos drivers nativos do **PostgreSQL** (`pg`) e **MySQL** (`mysql2`).

json

```
{
  "name": "db-designer-backend",
  "version": "1.0.0",
  "private": true,
  "main": "dist/server.js",
  "scripts": {
    "build": "tsc",
    "start": "node dist/server.js",
    "dev": "ts-node-dev --respawn --transpile-only server.ts"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.19.2",
    "knex": "^3.1.0",
    "mysql2": "^3.10.1",
    "pg": "^8.12.0"
  },
  "devDependencies": {
    "@types/cors": "^2.8.17",
    "@types/express": "^4.17.21",
    "@types/node": "^20.14.2",
```

Use o código com cuidado.

`tsconfig.json`

O arquivo de configuração do compilador TypeScript para gerar os arquivos de produção na pasta `dist/`.

json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "CommonJS",
    "lib": ["ES2022"],
    "outDir": "./dist",
    "rootDir": "./",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["server.ts"]
}
```

Use o código com cuidado.

---

4. Como rodar o ecossistema completo

Com todos os arquivos salvos em seus respectivos lugares (incluindo os códigos de `App.jsx`, `TableNode.jsx`, `server.ts` e os `Dockerfiles` criados nos passos anteriores), execute as seguintes etapas:

1. **Abra o terminal** na pasta raiz do projeto (`meu-modelador-web/`).
2. **Suba todo o ambiente** rodando o comando do Docker Compose:

Use o código com cuidado.

3. O Docker fará o download das imagens do PostgreSQL, MySQL, Nginx e Node, instalará todas as dependências isoladamente e iniciará os servidores.
4. Quando o terminal estabilizar, abra o seu navegador e acesse:
    1. **Interface Visual (Frontend):** `http://localhost`
    2. **API REST (Backend):** `http://localhost:3001`

A partir de agora, você tem um software de modelagem funcional rodando de ponta a ponta de forma 100% dockerizada.

O ambiente está totalmente desenhado e configurado! Se você for testar o projeto localmente, quer ajuda com mais alguma coisa, como:

- Um script inicial de inserção (**Seed**) de tabelas fictícias no banco de dados para o projeto já abrir com dados na Engenharia Reversa?
- Configurar variáveis de ambiente adicionais para produção?
- Ou prefere seguir de forma autônoma a partir deste ponto?